SECURITY CODE REVIEW AND ANALYSIS

1. PREPARATION

SUGGESTED READINGS

1. On code review in Mozilla: https://blog.humphd.org/vocamus-1569/?p=1569
2. OWASP code review guide: https://www.owasp.org/index.php/OWASP_Code_Review_Guide_Table_of_Contents
3. Static analysis tools: https://github.com/mre/awesome-static-analysis
4. Lessons from building static analysis tools at Google: https://cacm.acm.org/magazines/2018/4/226371-lessons-from-building-static-analysis-tools-at-google/fulltext

2. INTRODUCTION

The goal of this exercise is to make an introduction to performing source code review to find security vulnerabilities in web applications. The application that is going to be reviewed is a simple PHP application that allows users to upload and download files (you can think of it as a simple Dropbox app).

There are multiple reasons for doing code review:

- It can be faster than penetration testing. Some issues are really easy to spot during a code review (for example weak encryption), where others can take a lot more time (XSS for example).
- Compliance can require you to perform security code review (for example as part of PCI DSS 6.3.2).
- After doing penetration testing for a while; you want to do something different.
- You want to find better bugs. Some of the bugs you will find during a code review can be surprisingly hard to discover with black-box testing.
- You want to check if some code is backdoored (it's actually really hard to do).
- You want to write an exploit for a bug.

There are plenty of ways to perform code review. You can find some of the methods below:

- String matching/Grep for bugs.
- Following user input.
- Reading source code randomly.
- Read all the code.
- Check one functionality at a time (login, password reset...).

When doing a review, you need to look for everything:

- Weird behavior
- Missing checks
- Complexity
- Security checks already in place

- Differences between 2 functions/methods/classes
- Comparison and conditions (if/else)
- Regular expressions/string matching
- …

3. EXAMPLE

The application is a straightforward application with a dozen security issues.

As a user, you can only:

- Register/Login/Logout
- Upload and retrieve file

To get started you can get the code:

- Using git: git clone https://github.com/stojkovm/cr.git
- As a Zip file: https://github.com/stojkovm/cr/archive/master.zip

THE ISSUES

HARDCODED CREDENTIALS/SECRET

It's pretty easy to spot the hardcoded credentials or secrets if you read all the code. Here we can see that the following files contain some sensitive information:

- `classes/db.php`:

```
$lnk = mysql_connect("127.0.0.1", "root", "root");
```

- `classes/jwt.php`:

```
return hash("sha256","donth4ckmebr0".$data);
```

A modern web application should store all secrets outside of the source code of the application. During deployment, the secrets and credentials should be injected inside the application (via environment variables or configuration file). This limits access to production secrets/credentials for developers.

INFORMATION LEAK

To find this issue, you need to think about the way the application will get deployed. The web root of the application is the main directory of the source code. Therefore, the following files and directories will be publicly available:

- `.git`: this can be used to rebuild the full source code of the application.

- `deploy.sql`: this contains the database password in clear text and the hashed `admin` password.

These files should not be in the web root of the application.

SECURITY FLAGS ON COOKIES

The application use `setcookie` to set cookies for authentication. However, it doesn't set the `secure` and `httpOnly` flags on them in the following files:

- `login.php`:

```
setcookie("auth", User::createcookie($_POST['username'], $_POST['password']));
```

- `register.php`:

```
setcookie("auth", User::createcookie($_POST['username'], $_POST['password']));
```

WEAK PASSWORD HASHING MECHANISM

If you look at the `register` function in `classes/user.php`:

```
public static function register($user, $password) {
    $sql = "INSERT INTO  users (login,password) values (\"";
    $sql.= mysql_real_escape_string($user);
    $sql.= "\", md5(\"";
    $sql.= mysql_real_escape_string($password);
    $sql.= "\"))";
    $result = mysql_query($sql);
```

You can see that the application uses `md5` to hash passwords. Furthermore, the hashing is done without any seed. Finally, the hashing of the passwords is done by the database (as opposed to the application). This means that customers' password are likely to end up in the database logs or get transmitted in cleartext between the database and the application. The application should use scrypt, bcrypt or PBKDF2 to store the password.

CROSS-SITE SCRIPTING

The application uses the `h` function to escape data coming from the users. This function is just an alias for `htmlentities` defined in `classes/phpfix.php`:

```
function h() {
    return call_user_func_array("htmlentities", func_get_args());
```

```
  }
```

We can see that, `h` is not used in the following places:

- `index.php`:

```
<span class="text text-danger"><b><?php echo $error; ?></b></span>
```

- `login.php`:

```
<span class="text text-danger"><b><?php echo $error; ?></b></span>
```

- `register.php`:

```
<span class="text text-danger"><b><?php echo $error; ?></b></span>
```

However, `$error` is not under user's control. Therefore, those are only weaknesses, not vulnerabilities. They should still get fixed to future-proof the application.

The exploitable XSS in located in `index.php`:

```
<form action="<?php echo $_SERVER['PHP_SELF']; ?>" method="post" enctype="multipart/f
orm-data">
```

As `$_SERVER['PHP_SELF']` is user-controlled and can be used to trigger a XSS (using a path like `/index.php/"><script...`).

There is also another one in `classes/user.php` that can be triggered by registering multiple times the same username for example (due to the call to `mysql_error`).

NO CSRF PROTECTION

Here we can see that none of the forms are protected by a CSRF token. This is an example of the "what is missing" bugs, you can't see it because it's not there.

DIRECTORY LISTING

When users retrieve the list of their files the following code is called (in `classes/user.php`):

```
public static function getfiles($user) {
    $base = "files/".$user;
    if (!file_exists($base)) {
      mkdir($base);
```

```
    }
    return array_diff(scandir($base), array('..', '.'));
  }
```

This code gets the list of files (`scandir`) in `files/[USERNAME]` and removes the parent (`..`) and current (`.`) directory from the list using `array_diff`.

Nothing in this code (or during registration) prevents an attacker to use the following usernames: `..`, `../..`, `../../..` ... This will allow her to get a directory listing of any directory on the server hosting the application.

CRYPTO ISSUE

The crypto issue is located in `classes/jwt.php` in the signing function:

```
public static function signature($data) {
    return hash("sha256","donth4ckmebr0".$data);
  }
```

Here we can see that the application is using a hash instead of a hmac. This creates multiple issues:

- It's likely to be vulnerable to Length Extension
- It's not the RFC way to sign JWT token. This implementation will not work with other applications.

SIGNATURE BYPASS

The signature bypass is located in `classes/jwt.php`:

```
public static function verify($auth) {
    list($h64,$d64,$sign) = explode(".",$auth);
    if (!empty($sign) and (JWT::signature($h64.".".$d64) != $sign)) {
      die("Invalid Signature");
    }
```

We can see that the application get three elements from the `$auth` variable: `$h64`, `$d64` and `$sign`. Then it checks the signature `$sign` using: `JWT::signature($h64.".".$d64) != $sign`. However, it only checks the signature if one is provided: `if (!empty($sign) and...` . An attacker can provide a malicious `$auth` variable that doesn't contain a signature. This will allow him to bypass the signature mechanism.

AUTHORIZATION BYPASS

The authorization bypass is very simple. When a file is uploaded, it gets saved in the directory: `/files/[USERNAME]/[FILENAME]`:

```
public static function addfile($user) {
```

```
    $file = "files/".$user."/".basename($_FILES["file"]["name"]);
    if (!preg_match("/\.pdf/", $file)) {
      return  "Only PDF are allowed";
    } elseif (!move_uploaded_file($_FILES["file"]["tmp_name"], $file)) {
      return "Sorry, there was an error uploading your file.";
    }
```

It's trivial for an attacker to guess `[USERNAME]` and `[FILENAME]` to gain access to other users' files. The developer should ever put in place some access control or make the links impossible to guess.

REMOTE CODE EXECUTION

The RCE is located in the `addfile` function:

```
public static function addfile($user) {
    $file = "files/".$user."/".basename($_FILES["file"]["name"]);
    if (!preg_match("/\.pdf/", $file)) {
      return  "Only PDF are allowed";
    } elseif (!move_uploaded_file($_FILES["file"]["tmp_name"], $file)) {
      return "Sorry, there was an error uploading your file.";
    }
```

We already know that uploaded files are stored in the web root. We can see that the developer limits the file type users can upload to PDF. However, the regular expression is missing `$` to match the end of the string. It's possible for an attacker to upload a file named `exec.pdf.php`. As the file's extension is `.php` and the file is in the web root, it will get executed when accessed by the attacker.

ASSIGNMENT

1. Each student should select one project implemented in his/her past academic/professional experience and do manual code review and run the codebase against one of the tools from list (link no. 3).
2. Each student should write a report (max 3 pages) that include:
    a. Project Description: a high-level description of the project under review;
    b. Team: the list of the team members;
    c. Defects Found: A description of each fault found by each team member
    d. Summary of the Recommendation: a summary of the team recommendations to improve the code;
    e. Review Time and Defects Found: a summary of the effectiveness of each review. Specifically, the amount of time that each reviewer spent reviewing the code and the number of defects that each member identified.